

TechSaúde — Relatório Técnico de Desenvolvimento: Anexar Documento (Frente)

FATEC Barueri — Gestão de TI | Escopo: front-end de verificacao_documento.html (sem banco de dados/API)

1. Stack e arquitetura

Funcionalidade implementada dentro do arquivo único `verificacao_documento.html`, na seção `#screen-front` quando o modo ativo é "upload" (alternado pelo componente `.mode-toggle`). Não depende de bibliotecas externas: usa a API nativa `FileReader` do navegador para ler o arquivo selecionado. O trecho responsável corresponde a aproximadamente **20 linhas de HTML** (bloco `.dropzone`) e **35 linhas de JavaScript** (listeners de `click`, `dragover`, `dragleave`, `drop` e `change`, mais a função `handleFile()`).

2. Histórico de iterações

- **v1** — Definido, junto com o solicitante, que cada etapa de captura deveria oferecer o anexo de arquivo como alternativa equivalente à câmera, e não como recurso secundário escondido — daí o alternador `.mode-toggle` no topo da tela, com o mesmo peso visual para as duas opções.
- **v2** — Implementação da `.dropzone` com suporte a três formas de seleção: clique (abre o seletor nativo do sistema operacional via `input.click()`), arraste-e-solte (`dragover/drop`) e seleção direta pelo `<input type="file">`.
- **v3** — Adição de pré-visualização condicional: quando o arquivo é uma imagem (`file.type.startsWith("image/")`), a `dataURL` é exibida em `.dz-preview`; quando é um PDF, a pré-visualização de imagem é omitida (não há renderização de PDF embutida nesta camada).
- **v4** — Unificação com a etapa de revisão: tanto a foto tirada pela câmera quanto o arquivo anexado passam pelo mesmo componente `.review-row` ("Tirar novamente"/"Anexar outro" vs. "Usar esta foto"), mantendo consistência de UX entre os dois modos.

3. Funcionamento do anexo (frente)

1. Ao alternar para o modo "upload" em `#screen-front`, a câmera é interrompida (`stopStream("front")`, liberando o dispositivo de captura) e a `.dropzone` é exibida no lugar do visor de vídeo.
2. O usuário pode **clicar** na área tracejada (dispara `fileInput.click()`) ou **arrastar um arquivo** diretamente sobre ela; o evento `drop` extrai o arquivo de `e.dataTransfer.files[0]`.
3. Tipos aceitos, definidos no atributo `accept` do `input`: `image/*` e `application/pdf`; não há, nesta camada, validação adicional de tamanho máximo (o texto de apoio indica "até 10MB" como orientação visual, mas o limite não é aplicado via JavaScript).
4. O arquivo selecionado é lido com `FileReader.readAsDataURL()`; ao concluir (`reader.onload`), o resultado em base64 é guardado como `pendingDataURL` e, se for imagem, exibido em `.dz-preview`.
5. A barra de revisão é exibida (`.review-row.active`), com os mesmos botões da captura por câmera: **"Tirar novamente"** (neste modo, limpa o `input` e permite selecionar outro arquivo) e **"Usar esta foto"** (confirma e avança para a etapa de verso).

4. Envio e salvamento do documento (anexo — frente)

- O arquivo anexado **não é enviado a nenhum servidor** nesta camada: ao confirmar, o conteúdo lido pelo `FileReader` (já em formato `dataURL/base64`) é atribuído a `state.images.front`, exatamente no mesmo formato usado quando a foto é tirada pela câmera — o restante do fluxo (revisão, confirmação, miniaturas) **não diferencia a origem da imagem**.
- Não há upload real do arquivo original (`File object`) — apenas sua representação em `base64` é mantida em memória; o arquivo em si não é enviado a um servidor nem gravado em disco.
- Não há uso de `localStorage/sessionStorage`: o dado some ao recarregar ou fechar a aba.
- Se o arquivo for um PDF, o `dataURL` também é salvo em `state.images.front` (para eventual envio futuro), porém **não é exibido** como pré-visualização de imagem nem na tela de confirmação — a miniatura de "Frente" ficará em branco nesse caso, uma limitação conhecida desta versão.
- **Integração pendente (fora desta camada)**: validação server-side de tipo/tamanho real do arquivo, envio via `FormData/fetch()`, renderização de miniatura para arquivos PDF e persistência do arquivo original (não apenas sua versão em `base64`).

5. Acessibilidade (WCAG-aligned)

- Área de dropzone com texto explícito ("Toque para anexar ou arraste o arquivo aqui") em vez de depender apenas do ícone, para clareza em leitores de tela.
- Alternador de modo (`.mode-toggle`) implementado como botões (`<button>`) reais, navegáveis por teclado e com estado `.active` visualmente distinto.
- Pré-visualização de imagem com `alt="Pré-visualização do arquivo anexado"`.
- Input de arquivo nativo (`<input type="file">`), que herda automaticamente o suporte de acessibilidade do sistema operacional/navegador para seleção de arquivos.

6. Organização de assets

Nenhum asset externo: o ícone de upload da dropzone é um SVG inline; não há bibliotecas de terceiros para drag-and-drop (implementado com eventos nativos do DOM).

7. Testes executados

Verificação manual da seleção por clique, do arraste-e-solte de arquivos de imagem, e da alternância entre os modos câmera/anexo sem perda de estado da etapa. Não foi testado o comportamento com arquivos PDF reais em profundidade além da ausência de pré-visualização (comportamento esperado e documentado na seção 4). Sem testes automatizados implementados nesta rodada.

8. Fora de escopo

Validação real de tamanho máximo de arquivo, verificação de que o conteúdo do arquivo de fato corresponde a um documento (ex.: dimensões mínimas, nitidez), renderização de miniatura para PDFs e envio/persistência real do arquivo não estão implementados nesta etapa.